CM-EDU-001 / First steps

CogniCar Getting Started

Start the simulator, read feedback, and run your first program. All commands below run from the extracted `cognicar_sdk` directory.

1 Prepare your SDK

Use the Linux environment and system Python version specified in `docs/BUILD_INFO.txt`. Install the dependencies listed in `docs/REQUIREMENTS.md`. Use the ROS distribution recorded in the bundle; a DirectUdp-only bundle does not need ROS.

In **every new terminal**, including terminals used to build C++:

SDK environment

```
cd cognicar_sdk
source env.sh
python3 -c "import cognicar_interface, cogni_tracking"
```

No output from the import check means those modules loaded successfully.

2 Start one simulator

ROS bundle — viewer and controller together. In the first terminal:

Digital Twin

```
./bin/cognicar_visualization_ros
```

Press **Start**. Leave the manual speed and steering checkboxes disabled while a student program controls the car. Keep the window running.

DirectUdp-only bundle — headless simulation. Use this instead:

headless Digital Twin

```
./bin/cognicar_visualization --run-simulator
```

`--sim` connects the programs to it over loopback. A direct viewer and a separate controller cannot share the simulator feedback port.

> **What is simulated?** Pose, wheels, rear motors and IMU feedback are available. Servo feedback and LiDAR are not simulated. A missing scan is expected.

3 Check feedback

For a ROS bundle, open a second sourced terminal:

check the ROS connection

```
ros2 topic hz /state/odometry
```

A reported rate confirms state messages are arriving. Press Ctrl+C to leave the check.

4 Run your first drive

Start in simulation: the supplied driving examples take `--sim`; without it, it drives the real car. The supplied `drive` example waits for fresh front-wheel feedback, requests 150 rear-wheel RPM for three seconds, then requests a stop. Choose one language; run only one controller at a time.

Python 3

```
python3 exercises/drive.py --sim
```

C++17

```
cmake -S . -B build
cmake --build build --target drive -j2
./build/drive --sim
```

The simulated car should move straight, stop, and print `Test complete.` Ctrl+C ends the example early. `0_hello_car` is a separate exercise containing TODOs; use `drive` for the first working example.

5 Follow a path

Run the offline check before the closed loop:

Python 3

```
python3 exercises/1_path_tracking/check_tracker.py
python3 exercises/autopilot.py --sim
```

C++17

```
cmake --build build --target check_tracker autopilot -j2
./build/check_tracker
./build/autopilot --sim
```

Only launch `autopilot` after the checker passes and the simulator is running. The autopilot combines your path tracker and low-level speed controller. It sends steering in **radians** and rear motor currents in **amperes**.

Edit `exercises/1_path_tracking/student_tracker.py` or its `.hpp` counterpart. Replace one reference block at a time, run the checker, then repeat the simulation. Rebuild C++ after each change. The companion *CogniCar Path Tracking* guide explains the geometry and tuning.

6 Move to the real car

On a provisioned car with the bridge running, from the SDK root:

Real car: Python or built C++

```
python3 exercises/autopilot.py
./build/autopilot
```

The example starts LiDAR and built-in SLAM (mapping and localization). It waits for fresh feedback and a valid location estimate before driving. If feedback expires, it requests zero current and centred steering. Ctrl+C stops the program and LiDAR. In simulation, GPS feedback replaces SLAM.

A new map starts at the vehicle centre, facing map +x. Use that frame for your path. For a saved map, set `cfg.localization_map` and load a matching path. See `docs/LOCALIZATION.md`. Follow the lab's stop procedure.

7 Continue the lab

Location	What you use it for
`exercises/`	Editable C++ and Python programs and student blocks.
`include/`, `lib/`	Public C++ headers and prebuilt libraries.
`python/`	Importable Python bindings to the supplied libraries.
`bin/`	Simulator and runtime tools; contents depend on the bundle.
`data/`, `config/`	Reference paths and model/controller settings.
`docs/`	Build information, dependencies and bundled manuals.
`TOPICS.md`	Detailed ROS topic contract.

8 Other exercises

These checks run offline, without a car or ROS graph:

Python 3 checks

```
python3 exercises/2_state_estimation/check_kalman.py
python3 exercises/3_low_level_control/check_low_level_control.py
```

For C++, build and run the matching targets:

C++17 checks

```
cmake --build build --target check_kalman check_low_level_control -j2
./build/check_kalman
./build/check_low_level_control
```

The low-level controller converts target speed, wheel feedback, steering and IMU yaw rate into signed rear motor currents. Both languages have editable student blocks.

9 If something does not work

Symptom	First check
Import or library error	Source `env.sh`; use the Python and dependencies matched to this bundle.
No ROS telemetry	Press Start; check the bridge, network and `ROS_DOMAIN_ID`. Source the environment in this terminal.
No wheel feedback	Confirm the selected backend matches the running simulator or car. For direct UDP, check the address/interface and competing port users.
Car ignores the autopilot	Disable manual simulator controls; stop other command publishers. Check for missing or expired telemetry.
Tracker fails or oscillates	Restore the supplied settings, rerun the offline check, then change one block or gain at a time.

Next reference Use *CogniCar API* for C++/Python methods, telemetry freshness, units and ROS topics. Keep the SDK's `LICENSE.txt` with your installation.